DATA SCIENCE · MOCK INTERVIEW

Pack D — FinServe

Credit Scoring for the Unbanked

DOMAIN	PROBLEM TYPE	DURATION
Risk / Fintech	Binary Classification	55 minutes total

⚠️ **INTERVIEWER GUIDE —** Do not share this document with the candidate before or during the session.

Interview Flow

Block	Your Role	Time	Cumulative
Welcome	Ask: *"Tell me about yourself — what brings you here?"*	2 min	2 min
Live Coding	Share problems. Watch screen. Ask comprehension question verbally.	15 min	17 min
Case: Setup	Read the scenario aloud. Let them take notes.	3 min	20 min
Case: Drive	They drive. Reveal info only when asked. Challenge their choices.	25 min	45 min
Synthesis	*"Can you summarize your recommendation?"*	3 min	48 min
Q&A	They ask you questions. Improvise as a senior DS consultant.	5 min	53 min
Debrief	Score the rubric together. One strength, one area to improve.	10 min	63 min

Case Scenario

Opening Script *(Read Aloud — 2 minutes)*

> *"Our client is **FinServe**, a digital lending platform operating in Colombia, Mexico, and Peru. They offer microloans of $200–$2,000 through a mobile app, targeting underserved populations — small shop owners, gig workers, and people without formal banking relationships.*
>
> *Their problem: 60% of their loan applicants have no traditional credit history. The current scoring model — built on credit bureau data — simply rejects anyone without a credit score. That means they're turning away a massive market, including many creditworthy borrowers.*
>
> *FinServe has partnered with telecom providers and utility companies to access alternative data — mobile top-up patterns, utility payment history, mobile money transactions. They want to build a credit scoring model that can assess these 'thin-file' and 'no-file' applicants.*
>
> *Their goal: **approve 40% of currently rejected applicants while keeping the default rate below 8%.**"*

Information to Reveal — Only When Asked

> **Rule:** Never volunteer information. If they don't ask, don't tell.

If They Ask About…	What to Say
Current default rates	**Show Exhibit 1.** Full credit bureau applicants: 4.2% default. Alternative data only: 12.5%. Minimal data: 22%. Challenge is building a reliable model for the thin-file segments.
Alternative data / features	**Show Exhibit 2.** From a pilot of 5,000 applicants: mobile top-up frequency was the strongest predictor, then utility payment history, then mobile money transactions. Social media activity was nearly useless. Phone model tier was surprisingly predictive — it proxies for income level.
Loan economics	Avg loan: $800. Interest: 3.5%/month. Default = lose entire principal (no collateral). Break-even: 28 months of payments. Most loans are 6–12 months.
Current model	Simple logistic regression on credit bureau features. Binary accept/reject with a hard threshold. No granularity — a 61% probability of repayment gets the same 'reject' as 20%.

Data volume	200K approved loans (from the 40% with credit history). Alternative data pilot: only 5,000 observations with labeled outcomes.

Exhibits *(Screen-share at the right moment — only when asked)*

EXHIBIT 1 — Show when asked about default rates

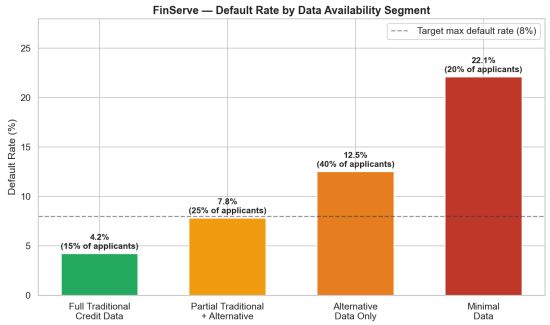
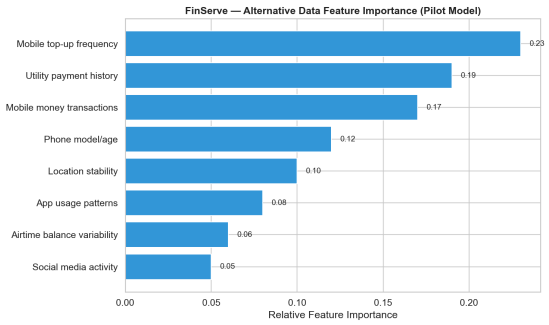


EXHIBIT 2 — Show when asked about alternative data



Challenges to Pose *(Pick 2–3 during the Structure phase)*

On small sample size

"You have 200K labeled observations with credit bureau data and only 5K with alternative data. How do you train a reliable model for the alternative data segment?"

On signal durability

"Mobile top-up frequency is predictive — but what if someone loses their job and stops topping up? How robust is this signal over time?"

On threshold setting

"FinServe wants to approve 40% of the currently rejected pool. How do you set the threshold? What's the tradeoff between default rate and approval rate?"

On validation

"You're scoring people who've NEVER had a loan. How do you validate the model without historical default data for this population?"

The Twist *(Drop around minute 18 of the case)*

DROP THIS AT 18:00

"One more thing — the financial regulator has just announced new rules requiring all credit models to demonstrate fairness. Specifically: the approval rate for women must be within 5 percentage points of the approval rate for men, and rural applicants must not be systematically disadvantaged. How does this affect your model?"

What a strong answer looks like: Model fairness is a real technical constraint, not just ethics. Options: (1) Fairness-aware training (equalized odds, demographic parity). (2) Post-processing — adjust thresholds by group. (3) Removing protected features doesn't help if proxies (region, phone model tier) encode them. (4) Regular fairness audits + monitoring. Key tension to name: fairness constraints may increase default rates — the tradeoff must be stated explicitly.

What a Strong Candidate Covers

1. **Reframes clearly:** *"How do we build a credit model that scores thin-file applicants reliably, approves 40% of them, keeps defaults under 8%, and meets fairness requirements?"*
2. **Two-model strategy or feature augmentation:** Credit bureau model for those who have it; alternative data model for thin-file applicants. Or a unified model using all available features.
3. **Addresses the small sample problem:** Only 5K labeled alternative data examples. Transfer learning from 200K traditional model? Semi-supervised? Active learning?
4. **Mentions calibration:** In lending, you need accurate probabilities, not just rankings. A well-calibrated model lets you set precise approval thresholds.
5. **Suggests risk-tiered pricing:** Don't just accept/reject — offer different loan terms (amount, rate, term) based on risk score.
6. **Handles fairness properly:** Doesn't just say "remove gender" — addresses proxies, uses fairness metrics, proposes monitoring.
7. **Quantifies impact:** 500K applicants × 60% rejected = 300K. Approve 40% = 120K new loans × $800 = **$96M in new lending**. At 8% default: $7.7M in losses. At 3.5%/mo over 9 months: $30M interest revenue. **Net: very positive.**

Timing Guide

Phase	Target Time	Notes

Read scenario aloud	0:00 – 2:00	
Reframe + their questions	2:00 – 7:00	Show exhibits when relevant
Their structure	7:00 – 20:00	Challenge at 12:00 and 16:00
Drop the twist	18:00	Watch how they adapt
Synthesis	22:00 – 25:00	*“Wrap up with a recommendation?”*

Coding Problems

Problem 1 — Applicant Risk Segmentation *(7 minutes)*

Dataset: `applicants.csv` — columns: `applicant_id`, `age`, `gender`, `region`, `has_bank_account`, `monthly_income_est`, `mobile_topup_freq`, `utility_payment_score`, `mobile_money_txns_monthly`, `phone_model_tier`, `location_changes_monthly`, `loan_amount_requested`, `defaulted`

Note: Some values may be missing.

Write a Python function that returns a DataFrame with one row per **region × has_bank_account**, containing:

```
1. region
2. has_bank_account (0 or 1)
3. applicant_count — Number of applicants
4. default_rate — % who defaulted (1 decimal)
5. avg_income — Avg estimated monthly income, nulls excluded (0 decimal)
6. avg_loan_requested — Avg loan amount requested (0 decimal)
7. avg_utility_score — Avg utility payment score, nulls excluded (1 decimal)

→ Sort by default_rate descending.
Bonus: Which segment has the highest default rate? Does it surprise you?
```

Problem 2 — Alternative Data Feature Analysis *(7 minutes)*

Using the same dataset, write a function that analyzes which **alternative data features** differ most between defaulters and non-defaulters.

For each of: `mobile_topup_freq`, `utility_payment_score`, `mobile_money_txns_monthly`, `location_changes_monthly`, `monthly_income_est` — calculate:

```
• mean_defaulted — Mean for applicants who defaulted (2 decimal)
• mean_non_defaulted — Mean for non-defaulters (2 decimal)
• pct_difference — (mean_defaulted - mean_non_defaulted) / mean_non_defaulted × 100
  (1 decimal)

Return columns: feature, mean_defaulted, mean_non_defaulted, pct_difference
Sort by absolute value of pct_difference descending. Drop nulls per feature.
Bonus: Which 2 features would you prioritize for a credit model, and why?
```

Code Comprehension — Conversational *(no coding required)*

Show this code. Ask the three questions below verbally.

```python
def build_credit_score(df, features, target='defaulted'):
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import roc_auc_score

    clean = df[features + [target]].dropna()
    X = clean[features]
    y = clean[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    model = LogisticRegression(class_weight='balanced')
    model.fit(X_train_scaled, y_train)

    y_prob = model.predict_proba(X_test_scaled)[:, 1]
    auc = roc_auc_score(y_test, y_prob)
    importance = dict(zip(features, model.coef_[0]))

    return model, scaler, auc, importance
```

Ask these three questions verbally:

1. **"Walk me through what this function does."**
   - Expected: Builds a logistic regression credit model. Drops rows with missing data, splits train/test, scales features, trains with balanced class weights (to handle default/non-default imbalance), evaluates with AUC, returns model + feature importances via coefficients.
2. **"What's the issue with `dropna()` on the full feature set?"**
   - Expected: If ANY feature has a missing value, the entire row is dropped. With multiple features having 5–8% missing, you could lose 20–30% of data. Better to impute, or use a model that handles missingness natively (e.g., gradient boosting).
3. **"The function uses `class_weight='balanced'`. What does that do, and when would you NOT want it?"**
   - Expected: Adjusts class weights inversely proportional to frequency — giving more weight to the minority class (defaulters). But in lending, the cost of a false positive (rejecting a good applicant) and a false negative (approving a defaulter) are asymmetric and dollar-valued. A cost-sensitive approach with explicit cost ratios may be more appropriate.

Interviewer Rules

→ **Screen sharing required** for coding. They can look up syntax — no AI tools.

→ **Never volunteer information.** If they don't ask, they don't get it.

- → **Challenge model choices.** “Why not a neural network?” “How would you explain this to the client’s risk committee?”
- → **Drop the fairness twist** at 18 minutes. Watch how they handle the ambiguity.
- → **Use the scoring rubric** — fill it out during or right after the session.
- → **Keep it conversational.** You’re a helpful colleague, not an examiner.

Make it real. The best thing you can do for your friend is not go easy on them. Challenge their assumptions, push back on vague answers, and give honest feedback during the debrief. That’s how they actually improve. 🏔️